

Material de Aula

Curso: Bacharelado em Ciência da Computação

Disciplina: Tecnologia de Orientação à Objetos (TOO)

Professora: Vanessa Lago Machado

Parte 2: Programação Orientada à Objetos (POO)

6. Métodos Especiais em Python (Dunder Methods)**

No Python, os **métodos especiais** (também chamados de **dunder methods**, por começarem e terminarem com `__`) permitem que possamos **personalizar o comportamento de objetos** em situações específicas.

Exemplos: inicialização, impressão, comparação, operações matemáticas, etc.

6.1. `__init__` Construtor da Classe

O método `__init__` é chamado **automaticamente** quando criamos uma nova instância de uma classe.

É nele que inicializamos os **atributos do objeto**.

Exemplo:

```
class Tarefa:
    def __init__(self, nome, descricao=None):
        self.nome = nome
```

```
        self.descricao = descricao
        self.concluida = False

# Criando um objeto
t1 = Tarefa("Aula T00", "Preparar material sobre dunder methods")
print(t1.nome)          # Aula T00
print(t1.descricao)     # Preparar material sobre dunder methods
```

- Equivalente ao **construtor** em outras linguagens como Java ou C#.

6.2. `__str__` - Representação em String

O método **str** define **como o objeto será exibido quando usamos print()** ou `str(objeto)`.

Sem ele, o Python mostra algo como `<main.Tarefa object at 0x...>`.

Exemplo:

```
class Tarefa:
    def __init__(self, nome):
        self.nome = nome
        self.concluida = False

    def __str__(self):
        return f"Tarefa: {self.nome} | Concluída? {self.concluida}"

# Testando
t1 = Tarefa("Estudar Python")
print(t1)
# Saída: Tarefa: Estudar Python | Concluída? False
```

- Facilita **debug** e deixa a saída **mais amigável para o usuário**.
- Equivalente ao **toString** em outras linguagens como Java, Javascript

6.3. `__eq__` – Comparação de Objetos

Por padrão, a comparação `==` em Python verifica se **dois objetos ocupam a mesma posição de memória**.

Com `__eq__`, podemos **definir nosso próprio critério de igualdade**.

Exemplo:

```
class Tarefa:
    def __init__(self, nome):
        self.nome = nome
        self.concluida = False

    def __eq__(self, outra):
        return self.nome == outra.nome and self.concluida == outra.concluida

# Testando
t1 = Tarefa("Estudar Python")
t2 = Tarefa("Estudar Python")

print(t1 == t2)    # True (mesmo nome e estado concluído)
print(t1 is t2)    # False (são objetos diferentes na memória)
```

- Com `__eq__`, podemos comparar objetos de forma **lógica** e não apenas pelo endereço de memória.
- Equivalente ao método **equals** em outras linguagens de programação, como Java.

Resumo dos Métodos Especiais em Python

Método	Nome/Significado	Função principal	Quando é chamado?
<code>__init__</code>	Construtor	Inicializa os atributos do objeto	Ao criar um novo objeto da classe
<code>__str__</code>	Representação em String	Retorna a forma legível do objeto (ex.: ao usar <code>print(obj)</code> ou <code>str(obj)</code>)	Quando o objeto é convertido para texto

Método	Nome/Significado	Função principal	Quando é chamado?
<code>__eq__</code>	Igualdade personalizada	Define como comparar dois objetos de acordo com critérios escolhidos	Quando usamos <code>obj1 == obj2</code>

Atividade Proposta

Implemente na classe **Tarefa** os seguintes métodos:

- O `__init__` deve inicializar os atributos.
- O `__str__` deve mostrar uma mensagem como: “Tarefa: Ir ao mercado (A FAZER)” ou “Tarefa: Atividade Metodos em TOO (CONCLUIDA)”.
- O `__eq__` deve considerar duas tarefas iguais **se tiverem o mesmo nome (titulo)**.

Teste seu código criando **duas tarefas diferentes com o mesmo nome** e comparando com `==`.